LSTM Assignment Report

Introduction

In this assignment, I used Long Short-Term Memory (LSTM) networks for two different tasks. First, I trained a model to classify Bangla text by sentiment. Then, I tested whether an LSTM trained on one time series could learn a second time series better than a new model trained from scratch. I used PyTorch for both experiments.

Part 1: Bangla Text Classification

For the first task, I used the **SentNoB** dataset. It contains Bangla text labeled as **Positive, Negative, or Neutral**. After cleaning, I had 12,570 training samples, 1,567 validation samples, and 1,586 test samples.

I removed URLs and unwanted punctuation, split each sentence into words, and converted the words into numerical IDs. I then padded the sequences so they could be processed together. Each message was limited to 60 tokens. I built the vocabulary from the training data to avoid using information from the test set.

My model had a trainable word embedding layer, an LSTM with 64 hidden units, dropout, and a final layer for the three sentiment classes. I trained it using the Adam optimizer. I allowed up to 30 epochs, but early stopping ended the saved run after 14 epochs. The model used the weights from epoch 8, which had the lowest validation loss.

Test results

Metric	Result
Accuracy	63.24%
Macro precision	59.22%
Macro recall	59.03%
Macro F1-score	58.64%

The notebook also shows accuracy and loss curves, a confusion matrix, and predictions for five Bangla sentences I tested. The model learned useful patterns, but it had trouble with some sentences, especially those labeled Neutral. One of my Neutral examples was classified as Positive.

The validation loss became worse after its best epoch. This suggests that training for more epochs would not necessarily improve the result; the model was starting to overfit the training data.

Part 2: Time-Series Transfer Learning

For the second task, I used **2,284 weekly CO_2 readings** as the source data and **252 monthly Pacific sea-surface temperature readings** as the target data. I used the previous 12 observations to predict the next value. I split the data by time rather than randomly, and I scaled each series using statistics from its training portion.

I first pretrained an LSTM with 32 hidden units on the CO_2 series. Next, I fine-tuned that model on the smaller temperature series. To make a comparison, I trained another LSTM with the same structure on the temperature series, starting with random weights. Both target models had the same maximum number of epochs, learning rate, and early-stopping settings.

Forecasting results

Model	RMSE (°C)	MAE (°C)
Fine-tuned LSTM	0.7685	0.6131
LSTM trained from scratch	**0.7224**	**0.5769**
Last-value baseline	1.1630	0.9710

Both LSTM models performed better than simply using the previous value as the next prediction. However, **the model trained from scratch performed better than the fine-tuned model**. Its RMSE and MAE were both lower. The actual-versus-predicted graph in the notebook helps show the difference.

One possible reason is that the two datasets are quite different. The source records CO_2 every week, while the target records sea-surface temperature every month. A window of 12 values therefore covers different amounts of time, and the patterns learned from CO_2 may not transfer well to temperature. I cannot confirm the exact cause from one training run.

Conclusion

I completed both LSTM tasks and evaluated them on test data. The Bangla classifier reached **63.24% accuracy**, though its errors show that there is room to improve. In the forecasting task, transfer learning did **not** improve the results: training from scratch gave the lower errors. This experiment helped me see that a pretrained model is not always the best choice. The match between the source and target datasets matters, and more tests with different random seeds would help confirm the result.